

Bitte beantworten Sie die Fragen in den dafür vorgesehenen Feldern. Wenn Sie keinen Platz mehr für eine Antwort haben, fahren Sie bitte auf der Rückseite fort und geben an wo es weiter geht.
Abgesehen von Internetfähigen Geräten sind alle Hilfsmittel zugelassen.
Schreiben Sie Ihre Klausur bitte nicht mit Bleistift und lassen Sie die Klausur bitte geheftet.
Sie haben 120 Minuten Zeit. Mit 53 Punkten gilt die Klausur als bestanden.

Vor- und Nachname (in Druckbuchstaben): _____

Matrikelnummer und Studiengang: _____

Aufgabe Nr.:	Punktzahl:	Davon erreicht:
1	14	
2	5	
3	5	
4	8	
5	10	
6	10	
7	10	
8	17	
9	8	
10	8	
11	14	
12	11	
Summe	120	

Note und Unterschrift des Erstprüfers: _____

Unterschrift des Zweitprüfers: _____

Lernraum I

1. (14 Punkte) Sie arbeiten an einer Anwendung, die die Aufgabenverwaltung für ein Team vereinfachen soll. Jedes Teammitglied hat eine eindeutige ID, einen Namen und eine Liste von Aufgaben, die ihm zugewiesen sind. Die Anwendung soll in der Lage sein, die folgenden vier Operationen effizient auszuführen:
 1. Einfügen eines neuen Teammitglieds mit all seinen Daten, einschließlich der Liste der zugewiesenen Aufgaben.
 2. Suchen eines Teammitglieds anhand seiner ID.
 3. Ersetzen der Liste der zugewiesenen Aufgaben für ein Teammitglied durch eine neue Liste.
 4. Löschen eines Teammitglieds aus der Aufgabenverwaltung.

Wählen Sie einen geeigneten abstrakten Datentyp und eine Datenstruktur, um die Teammitglieder und ihre Aufgaben effizient zu verwalten und die oben genannten Operationen optimal durchzuführen. Begründen Sie Ihre Wahl und erläutern Sie, warum dieser abstrakte Datentyp und diese Datenstruktur geeignet sind. Geben Sie zusätzlich die erwarteten Laufzeiten für die vier oben genannten Operationen in der O-Notation an.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

2. (5 Punkte) Bestimmen Sie die asymptotische Laufzeit der Funktion g in der Θ -Notation. Begründen Sie Ihre Antwort kurz.

```
void g(int n) {
    for (int i = 1; i <= n; i *= 2) {
        for (int j = 1; j <= n; j++) {
            // Führe eine konstante Anzahl von Operationen aus
        }
    }
}
```

[illegible]

3. (5 Punkte) Gegeben ist folgende Python-Funktion:

```
def recursive_function(n):
    if n <= 0:
        return 0
    return n + recursive_function(n - 1)
```

Stellen Sie die Rekursionsgleichung für die gegebene Funktion `recursive_function` auf. Verwenden Sie $T(n)$ als Symbol für die Laufzeit der Funktion für eine Eingabe der Größe n .

[illegible]

4. (8 Punkte) Geben Sie für die folgende Funktion `sort_me` die Θ -Notation für deren Laufzeit im Worst Case sowie die Θ -Notation für deren Laufzeit im Best Case in Abhängigkeit von n , der Größe des eindimensionalen Eingabe-Arrays `arr`, an. Welche Eigenschaft hat das Eingabe-Array `arr` im Worst Case und welche Eigenschaft im Best Case? Sie müssen nicht begründen wie Sie auf die Θ -Notation kommen.

```
def sort_me(arr):
    n = len(arr)
    for i in range(n):
        swapped = False # Flag, um Vertauschungen zu verfolgen
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True

        if not swapped:
            break

# Beispielaufruf
if __name__ == '__main__':
    arr = [4, 2, 7, 1, 9, 3]
    sort_me(arr)
    print("Liste:", arr)
```

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Lernraum II

5. Hashing mit Verkettung: Zur Speicherung von Zeichenketten in einer Hashtabelle der Größe 11 soll folgende Hashfunktion `hash_code` verwendet werden. Bei Kollisionen soll Hashing mit Verkettung genutzt werden. Hinweis: Der ASCII-Wert des Zeichens “a” beträgt 97, “b” entsprechend 98, etc. Dieser Wert wird von der Funktion `ord` zurückgegeben.

```
def hash_code(input_string):
    output = 1
    for c in input_string: # for each character c in the input_string
        output *= ord(c) - 95
    return output % 11
```

- (a) (6 Punkte) Tragen Sie die folgenden Zeichenketten in der angegebenen Reihenfolge in eine anfangs leere Hashtabelle ein. Tragen Sie dazu die Zeichenketten in die korrekten Slots der gegebenen Hashtabelle ein. Bitte geben Sie den Rechenweg an.

Zeichenketten: "abc", "ac", "cba", "acb", "ca"

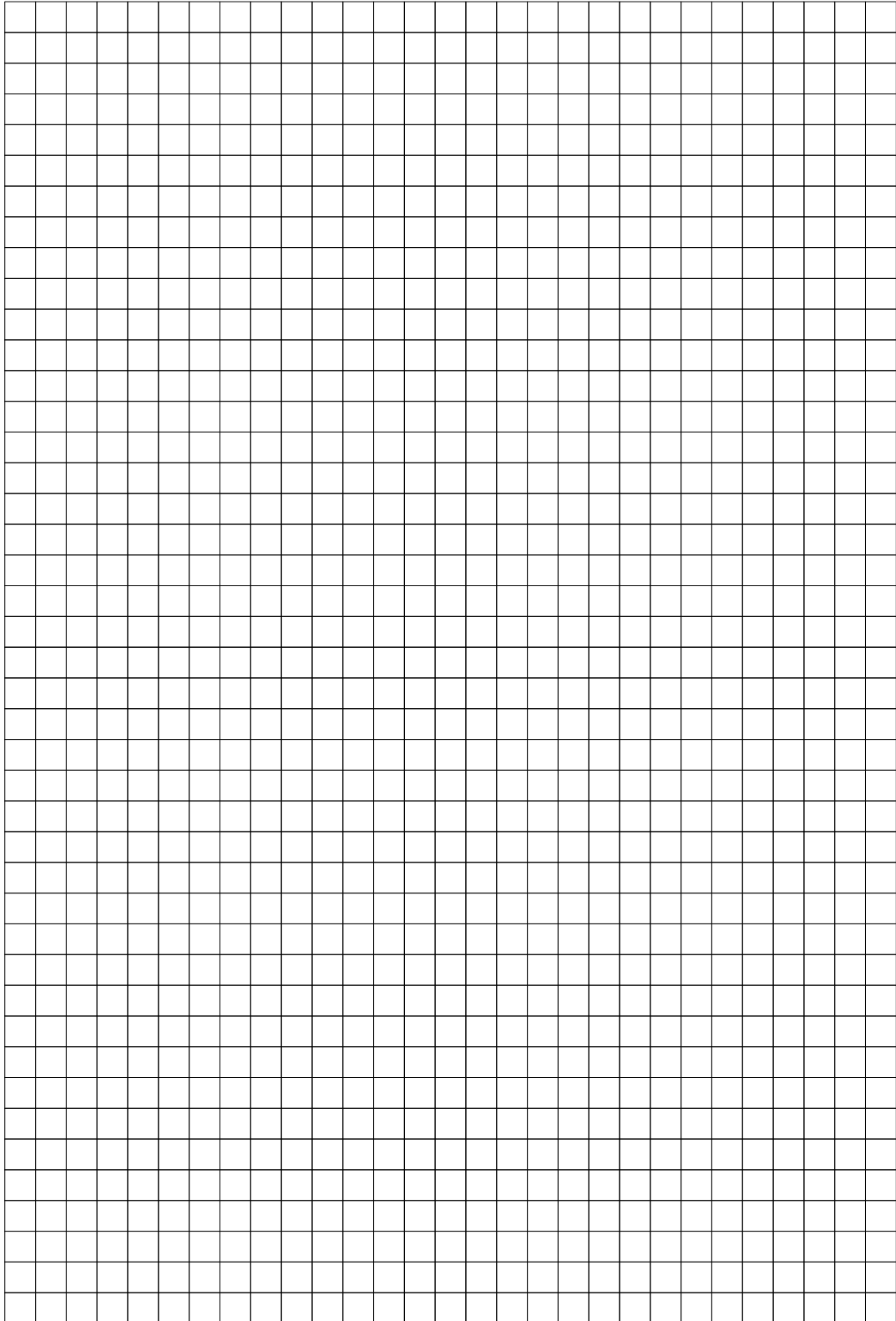
[illegible][illegible]

- (b) (4 Punkte) Geben Sie an, wie viele Operationen auf der Hashtabelle für die erfolglose Suche nach dem Schlüssel “cab” ausgeführt werden müssen. Auf welche Slots der Hashtabelle wird zugegriffen?

[illegible]

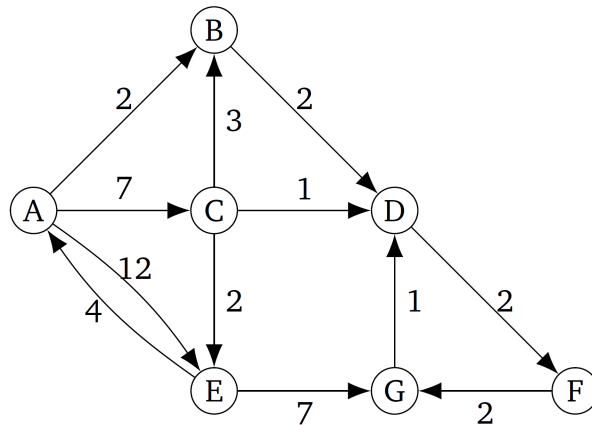
6. (10 Punkte) Fügen Sie die folgende Zahlenfolge in einen **Max**-Heap ein, indem Sie den buildHeap-Algorithmus von Floyd verwenden. Markieren Sie Ihre finale Lösung.

[10, 12, 20, 6, 7, 15, 17, 32]



Lernraum III

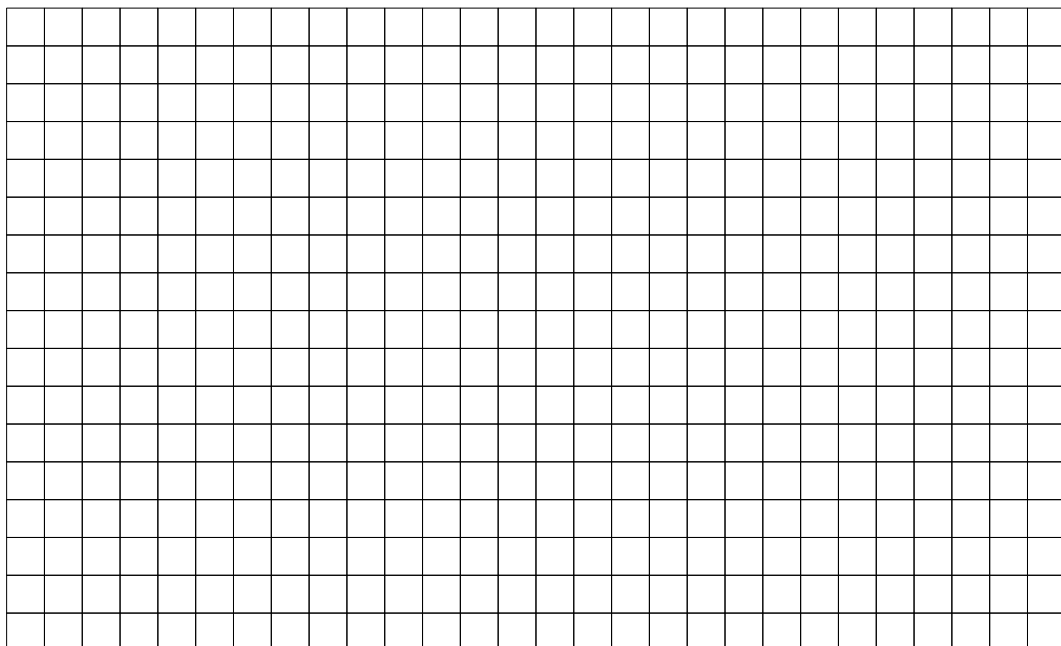
7. Kürzeste Pfade in einem gewichteten Graphen.



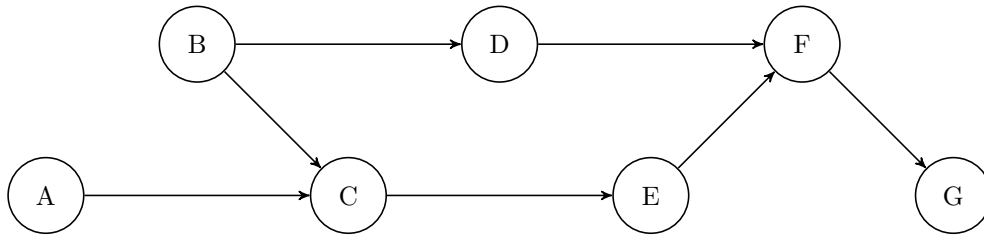
- (a) (7 Punkte) Bestimmen Sie den kürzesten Pfad vom Startknoten A aus zu allen anderen Knoten des obigen Graphen. Sie können dazu folgende Tabelle nutzen. Die kürzesten Abstände vom Knoten A aus sollen am Ende in der Spalte Distanz stehen.

Knoten	Distanz	Vorgänger	Verarbeitet?
A			
B			
C			
D			
E			
F			
G			

- (b) (3 Punkte) Zeichnen Sie den Baum kürzester Pfade von Startknoten A aus.



8. Gegeben ist folgender Graph.



- (a) (4 Punkte) Markieren Sie alle Lösungen, die eine korrekte topologische Sortierung des Graphen angeben.

○ B, A, C, E, D, F, G

○ A, B, C, D, E, F, G

○ B, D, A, C, E, F, G

○ A, B, D, C, E, F, G

- (b) (4 Punkte) Welche der obigen Lösungen sind weiterhin korrekt, wenn eine gerichtete Kante vom Knoten D zum Knoten C hinzugefügt werden würde?

.....

.....

.....

- (c) (5 Punkte) Geben Sie die Adjazenzmatrix des oben gezeichneten Graphen an, also ohne die zusätzliche Kante aus (b).

A full-page sheet of white graph paper featuring a uniform grid of thin black lines. The grid consists of small squares covering the entire area of the page.

- (d) (4 Punkte) Wie geht ein Algorithmus prinzipiell vor, der die Anzahl aller ausgehenden Kanten des Knoten B bestimmen soll, gegeben die Adjazenzmatrix?

.....

.....

.....

Lernraum IV

9. (8 Punkte) In einer Schule sollen die Schüler basierend auf ihren Noten in aufsteigender Reihenfolge sortiert werden. Die Schüler sind in Klassen organisiert, und jeder Schüler hat eine Klasse (z. B. Klasse 9A, Klasse 10B usw.). Die Noten der Schüler sind ganze Zahlen im Bereich von 0 bis 100.

Bei identischen Noten soll der Schüler aus einer unteren Klasse vor dem Schüler aus einer höheren Klasse einsortiert werden.

Diskutieren Sie für diese Anwendung, welche Sortieralgorithmen geeignet sind und welche nicht. Zur Auswahl stehen die folgenden Sortieralgorithmen:

1. Mergesort
2. Quicksort
3. Heapsort
4. Insertionsort
5. Selectionsort

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

10. (8 Punkte) Was ist die Best Case Laufzeit des Heapsort Algorithmus und begründen Sie Ihre Antwort. Geben Sie bspw. ein Beispiel an für das die Best Case Laufzeit gilt und erläutern Sie die Laufzeit jeden einzelnen Schritts des Algorithmus.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

11. (14 Punkte) Das Problem der längsten gemeinsamen Teilsequenz (longest common subsequence - LCS) besteht darin, die längste Teilsequenz zu finden, die in gegebenen zwei Sequenzen in derselben Reihenfolge vorhanden ist, d.h. die längste Sequenz zu finden, die aus der ersten Originalsequenz durch Löschen einiger Elemente und aus der zweiten Originalsequenz durch Löschen einiger Elemente erhalten werden kann.

Beispiel: Gegeben die beiden Sequenzen

X: CBAD

Y: CAB

Die Länge der LCS beträgt 2, da die LCS lautet: CA.

Die Brute-Force Lösung lautet:

```
# Funktion zum Ermitteln der Länge der längsten gemeinsamen Teilfolge von
# Sequenzen 'X[0m-1]' und 'Y[0n-1]'
def LCSLength(X, Y, m, n):

    # kehrt zurück, wenn das Ende einer der beiden Sequenzen erreicht ist
    if m == 0 or n == 0:
        return 0

    #, wenn das letzte Zeichen von 'X' und 'Y' übereinstimmt
    if X[m - 1] == Y[n - 1]:
        return LCSLength(X, Y, m - 1, n - 1) + 1

    # andernfalls, wenn das letzte Zeichen von 'X' und 'Y' nicht übereinstimmt
    return max(LCSLength(X, Y, m, n - 1), LCSLength(X, Y, m - 1, n))

if __name__ == '__main__':

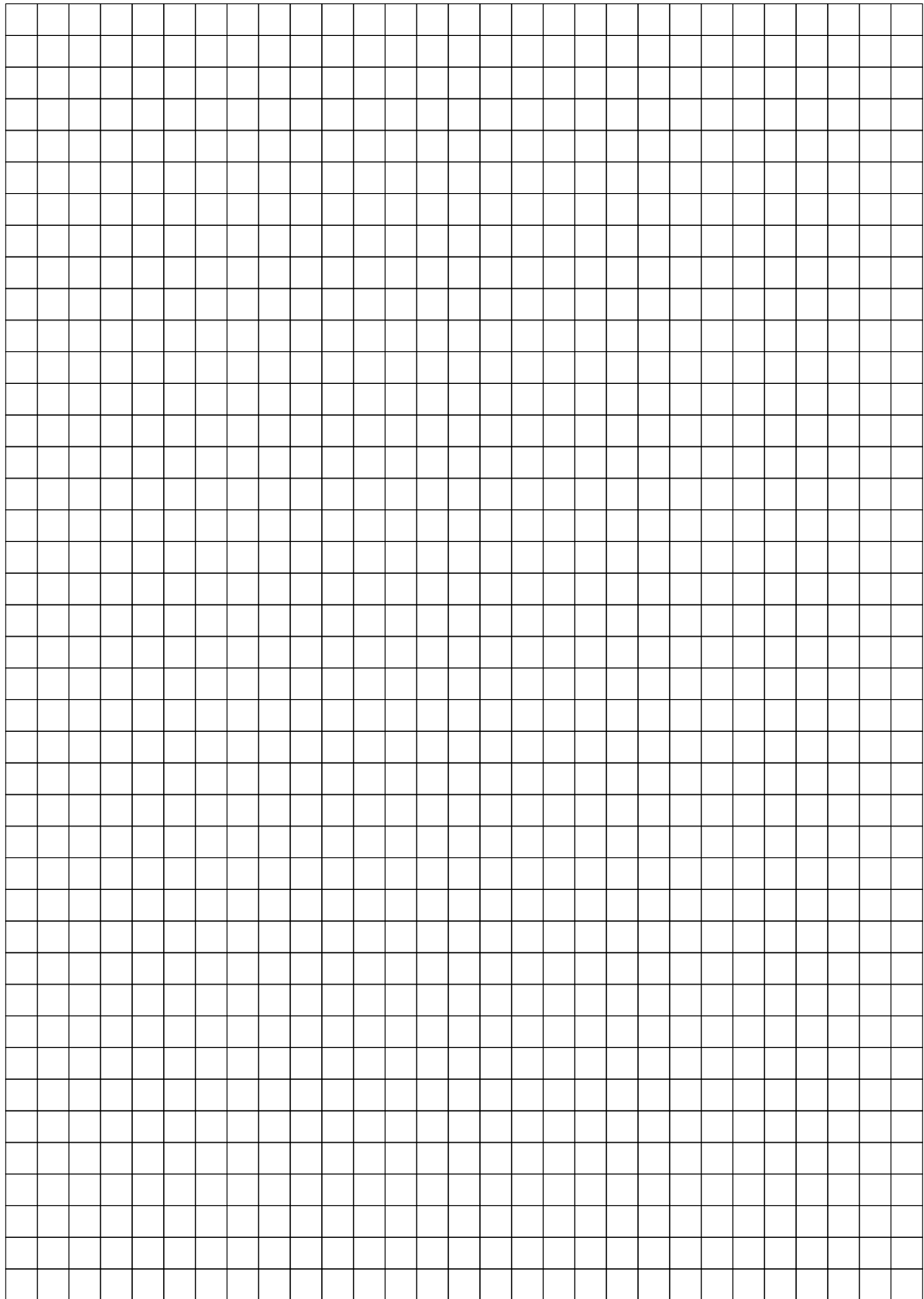
    X = 'CBAD'
    Y = 'CAB'

    print('The length of the LCS is', LCSLength(X, Y, len(X), len(Y)))
```

Zeigen Sie, dass dieser Code ineffizient ist, da die Funktion `LCSLength` häufig mit identischen Argumenten aufgerufen wird. Zeichnen Sie dazu den Rekursionsbaum der Methode `LCSLength` für das im Code aufgerufene Beispiel bis zur **maximalen Höhe von 3**. Damit Sie nicht so viel schreiben müssen, würde ich Ihnen empfehlen in die Knoten nur die beiden Argumente `m` und `n` einzuzichnen. Der Wurzelknoten des Rekursionsbaums sieht also so aus:

4, 3

Rechenkästchen folgen auf der nächsten Seite.



12. (11 Punkte) Schauen Sie sich die beiden Funktionen f1 und f2 an.

```
import random

f1(n):
    array = [k for k in range(1, n+1)]

    order_1 = [k for k in range(0, n)]
    order_2 = [k for k in range(0, n)]
    # after this call the digits in order_2 are in random order
    random.shuffle(order_2)

    sum = 0
    for i in range(n):
        sum += array[order_2[i]]

    print(sum)

f2(n):
    array = [k for k in range(1, n+1)]

    order_1 = [k for k in range(0, n)]
    order_2 = [k for k in range(0, n)]
    # after this call the digits in order_2 are in random order
    random.shuffle(order_2)

    sum = 0
    for i in range(n):
        sum += array[order_1[i]]

    print(sum)
```

Welche Funktion, f1 oder f2, wird in der Praxis für **sehr** große n (bspw. $n \geq 10^6$) schneller laufen? Begründen Sie Ihre Antwort und erläutern Sie eventuell genutzte Fachbegriffe.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....